Virtual Environments on ARC

Logistics

Please sign in:
https://docs.google.com/document/d/1jmMzTOsw9PACbf1_EDGZIEHL2Uj7TAlmXcEXH0LIWFA/edit?usp=sharing

Feedback form: https://forms.gle/P7BwENWMTpwHiH3bA

General Comments:

- Informal workshop so please feel free to interrupt me or use the chat for questions!
- This material is uploaded in Files on the Canvas site and in our Github Repo https://github.com/AdvancedResearchComputing/Workshops/blob/main/Virtual_Environments/VE_workshop.md
- I will post a recording of this workshop: https://docs.arc.vt.edu/usage/workshops.html
- We also have a lot of shorter video tutorials: https://docs.arc.vt.edu/usage/video.html#video
- If you want to follow along, make sure you are connected to VT network (VPN if off campus) and have an ARC account

Useful links:

- ARC's documentation site: https://docs.arc.vt.edu/
- GitHub Examples: https://github.com/AdvancedResearchComputing/examples
- Office Hours: https://arc.vt.edu/about/office-hours.html
- 4Help: https://arc.vt.edu/help
- Virtual Environments documentation page: https://docs.arc.vt.edu/usage/virtual_environments.html

Outline of this Workshop

1. Background & Motivation
2. Conda Virtual Environments: How to Build Using Miniforge/Miniconda
3. Using Virtual Environments and Jupyter Notebooks in OnDemand
4. Naming/Organizing VEs for different node types
5. Submitting jobs with Conda environments
6. Virtual Environments: How to Build Using `pip` and `venv` (if we have time)
7. Best practices & Wrap up

Background & Motivation

A virtual environment is a self-contained directory that contains a specific collection of installed packages and their dependencies.

Virtual environments (VE) on ARC can allow you to:

- run code with tailored environments and don't have to create them every time you want to run your code
- have more version control (python or other software)
- compile code/load modules outside of your $HOME environment

Most common types of VE are conda environments or using `pip` and `venv`.

Conda Virtual Environments: How to Build Using Miniforge/Miniconda

ARC suggests the use of Miniforge or Miniconda as the preferred way to build conda virtual environments (CVEs). ARC's provided Miniforge/Miniconda will work faster than user-installed anaconda in $HOME.

Steps for Building a Conda Virtual Environment (CVE)

1. Create an interactive session to a compute node.

```
interact --account=<account> --mem 8G --partition=v100_normal_q --
gres=gpu:1
```

2. Load Miniforge3/25.11.0-1.

```
module load Miniforge3/25.11.0-1
```

3. Create a CVE with a given name and this will be stored in `$HOME/.conda/envs/<name>`.

```
conda create -n falcon_v100_pytorch
```

OR you can create a CVE with a specified path and/or a specific python version

```
conda create -p ~/path/to/env/falcon_v100_pytorch python=3.12
```

4. Will have to type `y` to say yes to installing the listed packages:

```
...

The following NEW packages will be INSTALLED:

  _libgcc_mutex      conda-forge/linux-64::_libgcc_mutex-0.1-conda_forge
  _openmp_mutex      conda-forge/linux-64::_openmp_mutex-4.5-2_gnu
```

```
  bzip2               conda-forge/linux-64::bzip2-1.0.8-hda65f42_8
  ca-certificates     conda-forge/noarch::ca-certificates-2025.10.5-
hbd8a1cb_0
  ld_impl_linux-64    conda-forge/linux-64::ld_impl_linux-64-2.44-
ha97dd6f_2
  libexpat            conda-forge/linux-64::libexpat-2.7.1-hecca717_0
  libffi              conda-forge/linux-64::libffi-3.4.6-h2dba641_1
  libgcc              conda-forge/linux-64::libgcc-15.2.0-h767d61c_7
  libgcc-ng           conda-forge/linux-64::libgcc-ng-15.2.0-h69a702a_7
  libgomp             conda-forge/linux-64::libgomp-15.2.0-h767d61c_7
  liblzma             conda-forge/linux-64::liblzma-5.8.1-hb9d3cd8_2
  libnsl              conda-forge/linux-64::libnsl-2.0.1-hb9d3cd8_1
  libsqlite           conda-forge/linux-64::libsqlite-3.50.4-h0c1763c_0
  libuuid             conda-forge/linux-64::libuuid-2.41.2-he9a06e4_0
  libxcrypt           conda-forge/linux-64::libxcrypt-4.4.36-hd590300_1
  libzlib             conda-forge/linux-64::libzlib-1.3.1-hb9d3cd8_2
  ncurses             conda-forge/linux-64::ncurses-6.5-h2d0b736_3
  openssl             conda-forge/linux-64::openssl-3.5.4-h26f9b46_0
  pip                 conda-forge/noarch::pip-25.2-pyh8b19718_0
  python              conda-forge/linux-64::python-3.12.12-
hfe2f287_0_cpython
  readline            conda-forge/linux-64::readline-8.2-h8c095d6_2
  setuptools          conda-forge/noarch::setuptools-80.9.0-pyhff2d567_0
  tk                  conda-forge/linux-64::tk-8.6.13-noxft_hd72426e_102
  tzdata              conda-forge/noarch::tzdata-2025b-h78e105d_0
  wheel               conda-forge/noarch::wheel-0.45.1-pyhd8ed1ab_1


Proceed ([y]/n)? y
```

5. Activate the newly-created CVE.

```
source activate ~/path/to/env/<name of CVE>
```

6. Look at what packages are already installed in this VE.

```
conda list
```

7. Install package(s) to the CVE. You can install multiple packages (package names separated by commas).

```
conda install <package_name>
```

Sometimes, packages are not available via `conda install`. Alternatively, you may use `pip install`.

```
pip install <package_name>
```

8. Deactivate the CVE.

```
conda deactivate
```

9. Terminate the interactive session on the compute node.

```
exit
```

Using Virtual Environments and Jupyter Notebooks in OnDemand

Open On-Demand (OOD) is a web-based portal for using applications with a graphical interface on ARC systems. Many users use the Jupyter Notebook app on OOD and you can also use your VE's through OOD with a few modifications.

Follow these steps to use your VE in a Jupyter Notebook interactive session on OnDemand:

1. Check for `ipykernel` installation. Ensure that `ipykernel` is installed in your environment. After activating your environment, you can run the `pip freeze` command to list all the installed libraries. If `ipykernel` is not listed, you can install it by the following:

```
pip install ipykernel
```

2. Set up the environment 'Kernel'. Assuming your environment is named `falcon_v100_pytorch`, activate it and run the following command:

```
python -m ipykernel install --user --name falcon_v100_pytorch --display-name "Pytorch Falcon V100"
```

This will create a Jupyter Kernel named "Python (my_env)," which allows you to use 'falcon_v100_pytorch' within your notebooks. Note: you do not need to include the path of the virtual environment in the ipykernel, just the name.

3. Launch and select the Kernel in Jupyter Notebook. After launching Jupyter Notebook on OnDemand (ood.arc.vt.edu), open your notebook, then go to 'Kernel' in the top menu bar, select 'Change Kernel', and choose 'Pytorch Falcon V100' kernel. Remember to delete your OOD session when you are done using it, simply closing the browser WILL NOT stop the OOD job.

Naming/Organizing Virtual Environments for Different Node Types

Each cluster has at least two different node types. Each node type is equipped with a different cpu micro-architecture, slightly different operating system and/or kernel versions, slightly different system configuration and packages. All are tuned to be customized and efficient for the particular node features. These system differences can make virtual environments non-portable between node types.

As a result, you should create and build a virtual environment on a node of the type where you will use the environment.

Because you must build an environment for each node type, organizing your environments well is important for proper usage.

If given the `-n` flag, conda will create the environment in `$HOME/.conda/envs/<name>`. Environments stored in `$HOME/.conda/envs` are convenient becuase they can be loaded without the full path names and display nicely on the command prompt.

The `-p` flag may be used to specify a another location for your environment. This may be desirable if you want the environment in a shared location.

```
[user@tinkercliffs1 ~]$ module load Miniforge3/25.11.0-1
[user@tinkercliffs1 ~]$ conda env list
# conda environments:
#
base                     /apps/arch/software/Miniforge3/25.11.0-1
vis_tc_normal            /home/user/.conda/envs/vis_tc_normal
                         /projects/arcadm/graph_modeling/env_l40s/geoDL

[user@tinkercliffs1 ~]$ source activate vis_tc_normal
(vis_tc_normal) [user@tinkercliffs1 ~]$
```

However environments are organized, they should be used only for their corresponding node type.

Submitting jobs with Conda environments

If you want to follow along and also submit a job, you can do one of the following to obtain the scripts:

1. Download the slurm and python script from our Github repo example: https://github.com/AdvancedResearchComputing/examples/blob/master/python/miniconda/ and move them over to your ARC directory (either via scp or VS Code) Here is an example of a secure copy command from local computer to remote ARC systems:

```
scp numpy_compute.py example.slurm
<username>@tinkercliffs1.arc.vt.edu:/path/to/destination
```

OR

2. Copy the contents of the numpy_compute.py and example.slurm and make new files in your $HOME/user/ directory on ARC

OR

3. Git clone the full examples repo (easiest way):

```
git clone https://github.com/AdvancedResearchComputing/examples.git
cd examples/python/miniconda
```

We will have to modify the account name in the slurm script and then submit the job with the following commands:

```
sbatch example.slurm
squeue
```

Virtual Environments: How to Build Using `pip` and `venv`

ARC suggests the use of Miniforge/Miniconda as the preferred way to build conda virtual environments (CVEs).

However, there may be times when you want to go with a different approach to create a virtual environment (VE). One approach is using `pip` and `venv`. The approach is provided here. Note that the resulting VE is *not* a *conda* VE.

Steps for Building a Virtual Environment (VE)

1. Create an interactive session to a compute node.

```
interact --account=<account> --mem 8G --partition=v100_normal_q --
gres=gpu:1
```

2. Identify the Python versions available on the clusters.

```
module spider Python/3
```

```
-------------------------------------------------------------------------------
-----------------------------------------------------
  Python:
-------------------------------------------------------------------------------
-----------------------------------------------------
    Description:
      Python is a programming language that lets you work more quickly and
integrate your systems more effectively.

     Versions:
        Python/3.11.3-GCCcore-12.3.0
        Python/3.11.5-GCCcore-13.2.0
        Python/3.12.3-GCCcore-13.3.0
```

3. Load the desired Python version.

```
module load Python/3.12.3-GCCcore-13.3.0
```

4. Create the virtual environment (VE).

```
python -m venv ~/path/to/virt-env/<VE name>
```

5. Activate the VE. You must specify the `activate` script within the `bin` directory of your VE.

```
source ~/path/to/virt-env/<VE name>/bin/activate
```

6. Check the python version in the VE. You should get the same version of python as is in the module.

```
python --version
```

7. Install packages to your VE.

```
python -m pip install <package_name>
```

8. List the packages in the VE.

```
pip list
```

9. Deactivate the VE.

```
deactivate
```

10. Terminate the interactive session on the compute node.

```
exit
```

Wrap up

Virtual environments are great ways to create clean workflows on HPC systems like ARC.

- Each VE should be cluster/node specific to work properly so naming and organizing is VERY important
- Suggest using Miniforge/Miniconda for VEs (`module load Miniforge3`)
- If using VE with OOD Jupyter Notebooks, make sure `ipykernel` is install in conda environment
- Will have to `source activate` your VE in your slurm batch script

House Keeping

- Try to clean up `.conda/pkgs` directory in `$HOME/user`

```
conda clean --all
```

- Delete unused virutal environements

```
conda env remove -n <environment_name>
```

OR if the VE is in a specific location other than `$HOME/.conda/envs`

```
conda env remove -p /full/path/to/your/environment
```

Debugging

Sometimes if `source activate <environment_name>` doesn't work, try running the following:

```
eval "$(conda shell.bash hook)"
```

then activate your environment:

```
source activate <environment_name>
```

verify that it is loaded correctly with your python version by running

```
which python
```

and this should point to the python version within your environment.